

American University of Sharjah

College of Engineering

COE 420L – Software Engineering Lab

Lab 6: UML Sequence Diagram Modeling

Project: Lost and Found Property Claim System

Team Members:
Hamda Albahri — g00100284
Hamda Bahakeem — g00100146
Mahra Alhammadi — g00098713

GitHub Repository:
https://github.com/COE420-Group1-F26/LostandFound-System

Date of Submission: November 2, 2026

Team Contributions:

Each member drew the sequence diagram for one of the selected use cases and worked on a separate part of the lab. Every member's work can be seen in the commit history of the GitHub repository (each commit message includes the member's name).

Task	**Mahra Alhammadi (g00098713) — Member 1**	**Hamda Albahri (g00100284) — Member 2**	**Hamda Bahakeem (g00100146) — Member 3**
Exercise 1 – Selected use cases (Selected_Use_Cases.md)	Yes	—	—
Exercise 2 – UC-03 Search Found Items sequence diagram	Yes	—	—
Exercise 2 – UC-09 Generate Match Suggestions sequence diagram	—	Yes	—
Exercise 2 – UC-14 Resolve Disputed Claim sequence diagram	—	—	Yes
Sequence diagram consistency check	Yes	—	—
Class diagram update and Lab5_Changes.md	—	Yes	—
Final report and submission	—	—	Yes
Final check of the report	Yes	Yes	Yes

Exercise 1 – Selected Use Cases:

Three major use cases were selected from Project/Requirements/Use_Cases.md. Each one comes from a different part of the system (search, matching and notification, claims), each is supported by the classes in the Lab 5 class diagram, and each includes repeated and conditional steps.

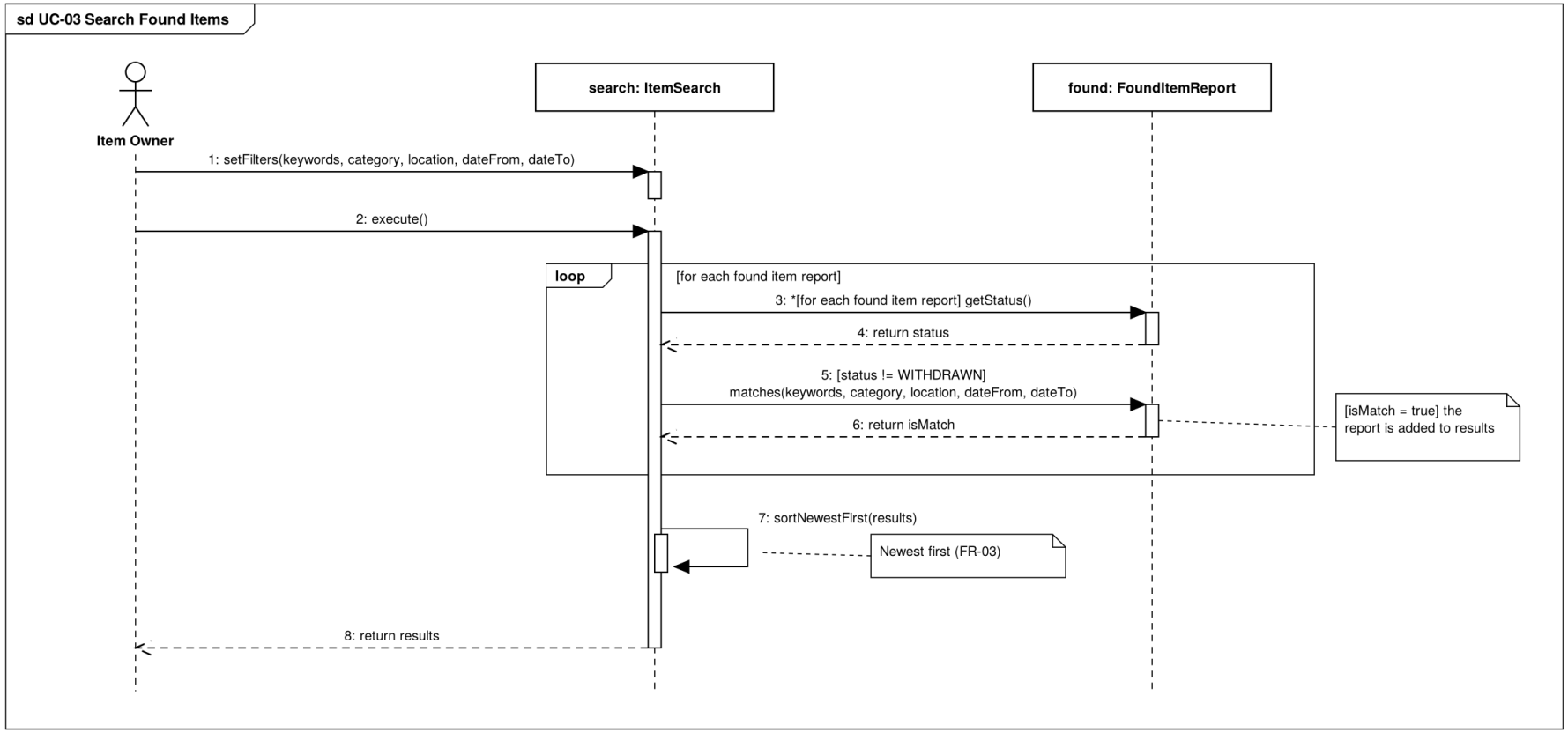
Use Case ID	**Use Case Name**	**Primary Actor**	**Purpose**	**Reason for Selection**
UC-03	Search Found Items	Item Owner	The owner searches and filters the found items to check whether their item has been handed in.	The main way an owner finds their item (FR-03). The search checks every found report, skips withdrawn ones and orders the results, so it shows iteration and a guard. Diagram drawn by Member 1.
UC-09	Generate Match Suggestions	Finder (secondary: Email / Notification Service)	The system compares a newly submitted found report with the lost reports, stores ranked match suggestions and emails the owner of each matching lost report.	The core automatic feature of the system (FR-04, FR-09). It involves the matching control object, four entity classes and the external email service, with iterations over the lost reports and the suggestions and a minimum-score guard. Diagram drawn by Member 2.
UC-14	Resolve Disputed Claim	Campus Security Officer	The officer compares the competing claims for the same found item, approves the rightful owner’s claim and rejects all the others.	Covers the claim process end to end: it includes UC-11, UC-12 and UC-13 (FR-11 to FR-13, FR-16), records the claim history (FR-15) and changes the item status. Diagram drawn by Member 3.

Exercise 2 – UML Sequence Diagrams:

Each member drew one sequence diagram in draw.io. Every diagram starts with the actor in the first column, uses the objects and operations from the Lab 5 class diagram, and shows numbered messages, dashed return messages, activation bars, at least one iteration (*) inside a loop frame and at least one guard ([ ]). Each diagram is followed by its participants and messages.

UC-03 Search Found Items (Mahra Alhammadi, g00098713):

The item owner first sets the search filters on the ItemSearch control object and then runs the search. ItemSearch goes through every found item report in a loop and checks its status. Withdrawn reports are skipped (guard), and every other report is asked whether it matches the filters. The matching reports are sorted newest first and returned to the owner (FR-03).



Use case: UC-03 Search Found Items

Primary actor: Item Owner

Requirements: FR-03

Participants:

Lifeline	Class	Object type
Item Owner	Actor (stick figure)	The person; outside the system
search: ItemSearch	ItemSearch	Control object (2nd column)
found: FoundItemReport	FoundItemReport (inherits ItemReport)	Entity object; one lifeline stands for every found report

Messages:

No.	From → To	Message	Class operation
1	Item Owner → search	setFilters(keywords, category, location, dateFrom, dateTo)	ItemSearch.setFilters(...): void (added in Lab 6)
2	Item Owner → search	execute()	ItemSearch.execute(): List<FoundItemReport>
3	search → found	*[for each found item report] getStatus()	FoundItemReport.getStatus(): ReportStatus
4	found → search (dashed)	return status	return message
5	search → found	[status != WITHDRAWN] matches(keywords, category, location, dateFrom, dateTo)	ItemReport.matches(...): boolean (added in Lab 6, inherited)
6	found → search (dashed)	return isMatch	return message
7	search → search (self-call)	sortNewestFirst(results)	ItemSearch.sortNewestFirst(...) (private, added in Lab 6)
8	search → Item Owner (dashed)	return results	return List<FoundItemReport>

Iteration and guards: iteration on message 3; guard on message 5. Messages 3 to 6 are inside a loop frame (for each found item report).

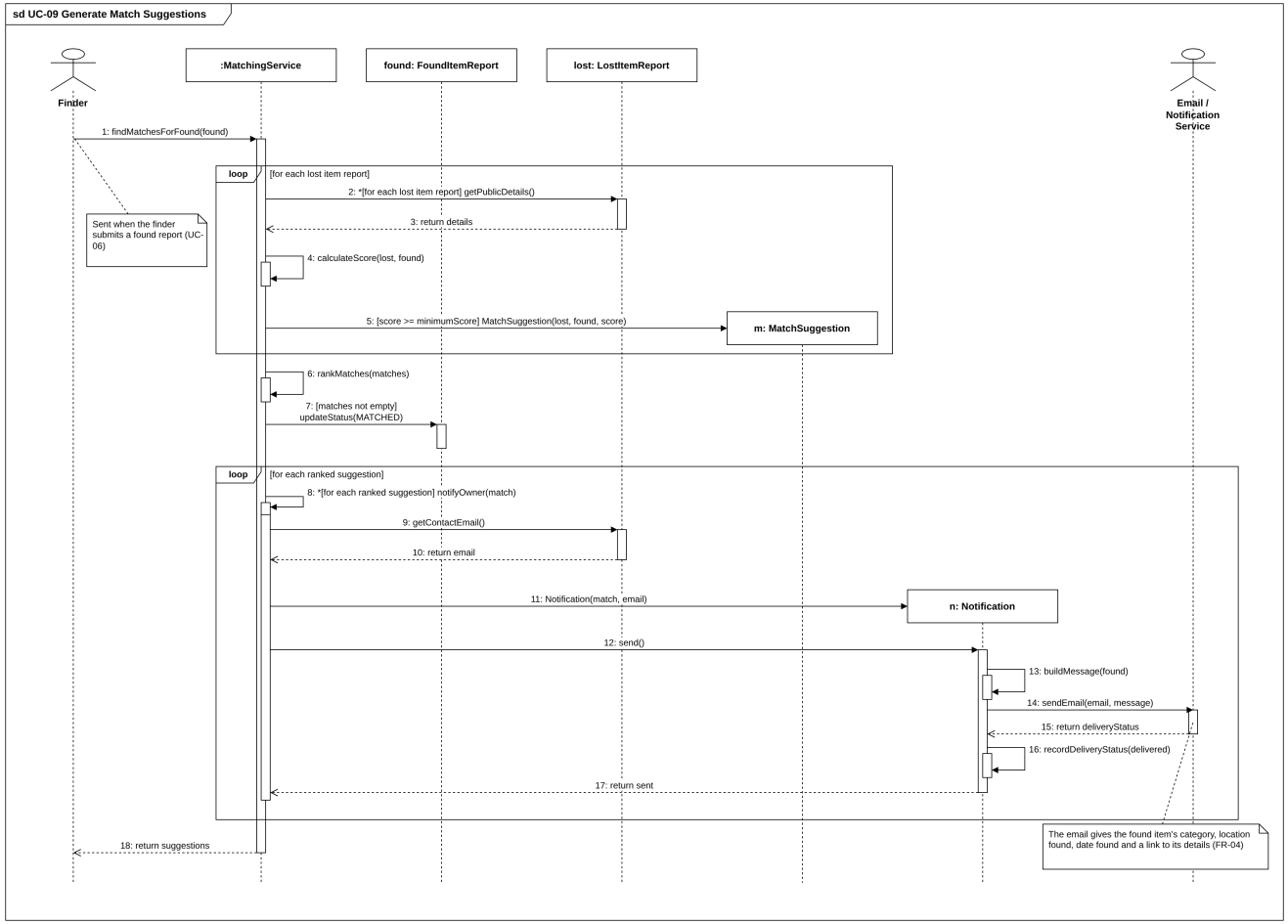
Notes in the diagram: [isMatch = true] the report is added to results; results are sorted newest first (FR-03).

Operations this diagram needs that were not in the Lab 5 class diagram:

Class	Operation	Reason
ItemSearch	+ setFilters(keywords: String, category: String, location: String, dateFrom: Date, dateTo: Date): void	Message 1: the owner enters the search filters (FR-03).
ItemSearch	- sortNewestFirst(results: List<FoundItemReport>): List<FoundItemReport>	Message 7: results are shown newest first (FR-03).
ItemReport	+ matches(keywords: String, category: String, location: String, dateFrom: Date, dateTo: Date): boolean	Message 5: each found report says whether it fits the filters (inherited by FoundItemReport).
CampusMember	Remove + startSearch(): ItemSearch and the CampusMember ⇢ ItemSearch dependency	The owner works with the ItemSearch control object directly; entity objects should not access control objects (lecture slides).

UC-09 Generate Match Suggestions (Hamda Albahri, g00100284):

When a finder submits a found report, MatchingService loops over the lost item reports, gets each one's public details and calculates a score. A MatchSuggestion is created only when the score reaches the minimum score (guard). The suggestions are ranked and, if there is at least one, the found item becomes MATCHED. For each ranked suggestion, the service gets the owner's contact email from the lost report, creates a Notification and sends it through the external email service, which returns the delivery status (FR-04, FR-09).



Use case: UC-09 Generate Match Suggestions

Primary actor: Finder (secondary: Email / Notification Service)

Requirements: FR-04, FR-09

Participants:

Lifeline	Class	Object type
Finder	Actor (stick figure), 1st column	The person who submitted the found report
:MatchingService	MatchingService	Control object (2nd column)
found: FoundItemReport	FoundItemReport	Entity object
lost: LostItemReport	LostItemReport	Entity object; one lifeline stands for every lost report
m: MatchSuggestion	MatchSuggestion	Entity object, created during the interaction
n: Notification	Notification	Entity object, created during the interaction
Email / Notification Service	Actor (stick figure), last column	External system; outside the system

Messages:

No.	From → To	Message	Class operation
1	Finder → :MatchingService	findMatchesForFound(found)	MatchingService.findMatchesForFound(found: FoundItemReport): List<MatchSuggestion>
2	MatchingService → lost	*[for each lost item report] getPublicDetails()	LostItemReport.getPublicDetails(): String
3	lost → MatchingService (dashed)	return details	return message
4	MatchingService → MatchingService (self-call)	calculateScore(lost, found)	MatchingService.calculateScore(...): double (private)
5	MatchingService → m (creates m)	[score >= minimumScore] MatchSuggestion(lost, found, score)	creation (constructor)
6	MatchingService → MatchingService (self-call)	rankMatches(matches)	MatchingService.rankMatches(...): List<MatchSuggestion> (private)
7	MatchingService → found	[matches not empty] updateStatus(MATCHED)	FoundItemReport.updateStatus(newStatus: ReportStatus): void
8	MatchingService → MatchingService (self-call)	*[for each ranked suggestion] notifyOwner(match)	MatchingService.notifyOwner(match: MatchSuggestion): Notification
9	MatchingService → lost	getContactEmail()	LostItemReport.getContactEmail(): String (added in Lab 6)
10	lost → MatchingService (dashed)	return email	return message
11	MatchingService → n (creates n)	Notification(match, email)	creation (constructor)
12	MatchingService → n	send()	Notification.send(): boolean
13	n → n (self-call)	buildMessage(found)	Notification.buildMessage(found: FoundItemReport): String (private)
14	n → Email / Notification Service	sendEmail(email, message)	external system (not a class operation)
15	Email / Notification Service → n (dashed)	return deliveryStatus	return message (DFD flow "Delivery Status")
16	n → n (self-call)	recordDeliveryStatus(delivered)	Notification.recordDeliveryStatus(delivered: boolean): void
17	n → MatchingService (dashed)	return sent	return message
18	MatchingService → Finder (dashed)	return suggestions	return List<MatchSuggestion>

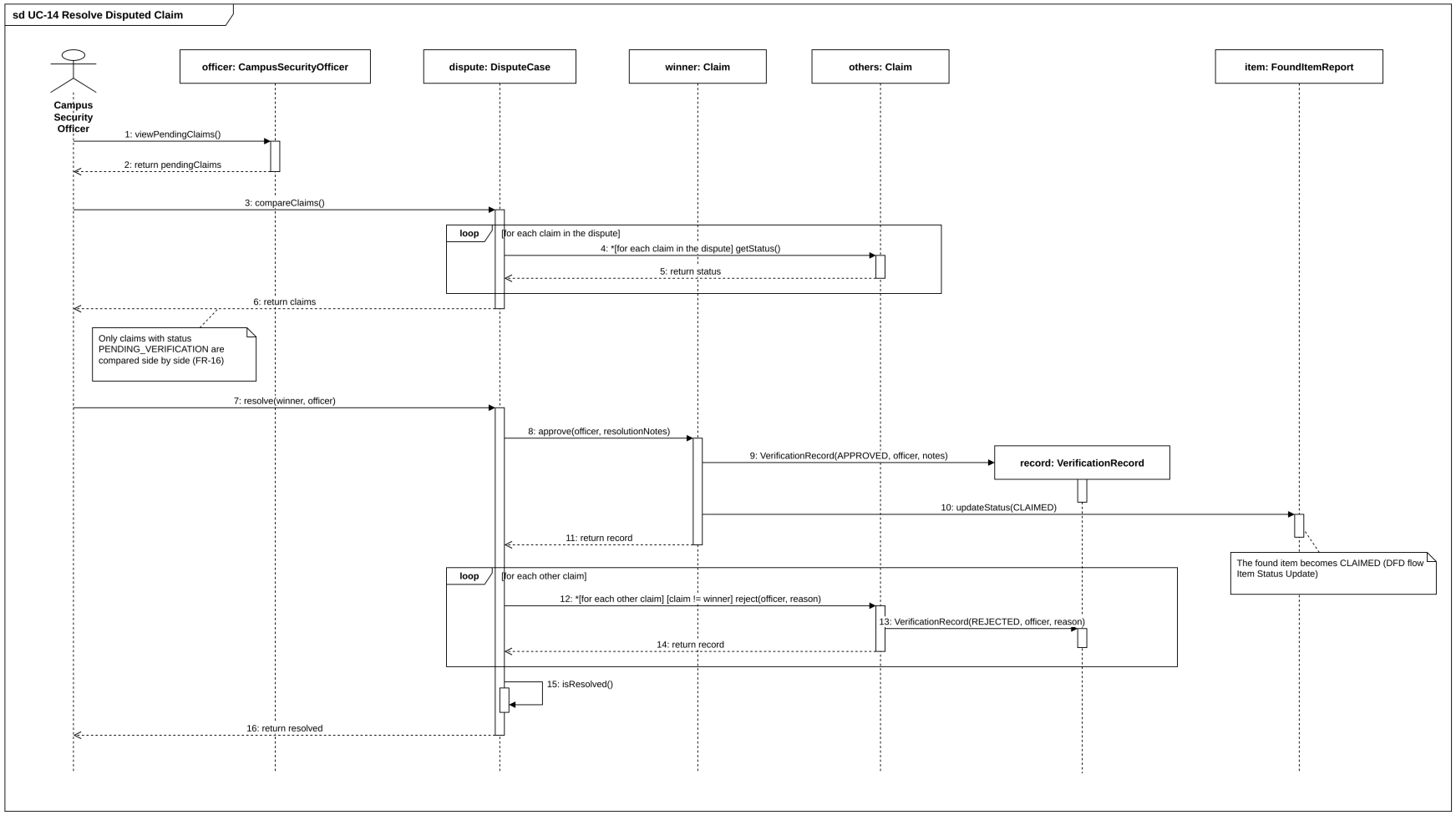
Iteration and guards: iteration on messages 2 and 8; guards on messages 5 and 7.

Operations this diagram needs that were not in the Lab 5 class diagram:

Class	Operation	Reason
LostItemReport	+ getContactEmail(): String	Message 9: the notification copies the owner's contact email from the lost report into recipientEmail (Lab 5 decision, D2).

UC-14 Resolve Disputed Claim (Hamda Bahakeem, g00100146):

The officer first views the pending claims and then asks the DisputeCase to compare the competing claims, so the dispute loops over its claims and checks each one's status. The officer then resolves the dispute with the winning claim. The winner is approved, which creates an APPROVED VerificationRecord and changes the found item to CLAIMED. The dispute then loops over every other claim and rejects it (guard: claim != winner), creating a REJECTED record for each one. Finally the dispute checks that it is resolved and returns the result to the officer (FR-11 to FR-16).



Use case: UC-14 Resolve Disputed Claim

Primary actor: Campus Security Officer

Requirements: FR-16 (with FR-11 to FR-15)

Participants:

Lifeline	Class	Object type
Campus Security Officer	Actor (stick figure)	The person; outside the system
officer: CampusSecurityOfficer	CampusSecurityOfficer	Entity object (the officer's account)
dispute: DisputeCase	DisputeCase	Entity object
winner: Claim	Claim	Entity object (the claim that is approved)
others: Claim	Claim	Entity object; one lifeline stands for every other claim
record: VerificationRecord	VerificationRecord	Entity object, created during the interaction
item: FoundItemReport	FoundItemReport	Entity object

Messages:

No.	From → To	Message	Class operation
1	Officer (actor) → officer	viewPendingClaims()	CampusSecurityOfficer.viewPendingClaims(): List<Claim> (UC-11)
2	officer → Officer (dashed)	return pendingClaims	return List<Claim>
3	Officer → dispute	compareClaims()	DisputeCase.compareClaims(): List<Claim>
4	dispute → others	*[for each claim in the dispute] getStatus()	Claim.getStatus(): ClaimStatus
5	others → dispute (dashed)	return status	return message
6	dispute → Officer (dashed)	return claims	return List<Claim>
7	Officer → dispute	resolve(winner, officer)	DisputeCase.resolve(approved: Claim, officer: CampusSecurityOfficer): void
8	dispute → winner	approve(officer, resolutionNotes)	Claim.approve(officer, notes): VerificationRecord (UC-12)
9	winner → record (creates record)	VerificationRecord(APPROVED, officer, notes)	creation (constructor)
10	winner → item	updateStatus(CLAIMED)	FoundItemReport.updateStatus(newStatus: ReportStatus): void
11	winner → dispute (dashed)	return record	return VerificationRecord
12	dispute → others	*[for each other claim] [claim != winner] reject(officer, reason)	Claim.reject(officer, reason): VerificationRecord (UC-13)
13	others → record (creates record)	VerificationRecord(REJECTED, officer, reason)	creation (constructor)
14	others → dispute (dashed)	return record	return VerificationRecord
15	dispute → dispute (self-call)	isResolved()	DisputeCase.isResolved(): boolean
16	dispute → Officer (dashed)	return resolved	return message

Iteration and guards: iteration on messages 4 and 12; guard on message 12. Messages 4 to 5 and messages 12 to 14 are inside loop frames.

Notes in the diagram: only claims with status PENDING_VERIFICATION are compared side by side (FR-16); the found item becomes CLAIMED (DFD flow Item Status Update).

Operations this diagram needs that were not in the Lab 5 class diagram:

Class	Operation	Reason
—	None	Every message is an operation already in the Lab 5 class diagram, a return, or a constructor.

Exercise 3 – Sequence Diagram Consistency Verification:

Checked by: Mahra Alhammadi (g00098713)

Diagrams checked: UC-03 Search Found Items (Member 1), UC-09 Generate Match Suggestions (Member 2), UC-14 Resolve Disputed Claim (Member 3), in Project/Behavioral_Modeling/Sequence_Diagrams. Each diagram was compared with Project/Requirements/Use_Cases.md, Functional_Requirements.md, the Level-0 DFD and the updated class diagram (Project/Object_Modeling/Class_Diagram). The message labels in each .drawio file were checked against the message table in its notes file.

A. Consistency with Lab 3 — Requirements and Use Cases:

Check	UC-03	UC-09	UC-14	Evidence
The diagram represents one of the approved use cases	Yes	Yes	Yes	The title frame of each diagram names UC-03, UC-09 and UC-14, which are in Use_Cases.md and Selected_Use_Cases.md.
The interaction supports the use case's goal	Yes	Yes	Yes	UC-03 returns the matching found items to the owner (message 8). UC-09 stores ranked match suggestions and emails the owner of each lost report (messages 5, 6, 14). UC-14 approves one claim and rejects every other claim for the item (messages 8, 12).
The sequence reflects the use case's description and flow	Yes	Yes	Yes	UC-03: keyword, category, location and date filters, results newest first (messages 1, 5, 7; FR-03). UC-09: compares the found report with the lost reports and emails the owner of the lost report; the finder gets the suggestions back (messages 2–5, 11–14, 18; FR-04, FR-09). UC-14: views pending claims, compares the competing claims, approves one and rejects the rest, as in its includes of UC-11, UC-12 and UC-13 (messages 1, 3–6, 8, 12; FR-16).
The messages do not introduce unsupported functionality	Yes	Yes	Yes	Every message traces to FR-03 (UC-03), FR-04 and FR-09 (UC-09), or FR-11 to FR-16 (UC-14). No diagram edits, deletes or pays for anything that the requirements do not allow.

B. Consistency with Lab 5 — UML Class Diagram:

Check	UC-03	UC-09	UC-14	Evidence
Every participating object corresponds to a class in the class diagram	Yes	Yes	Yes	UC-03: ItemSearch, FoundItemReport. UC-09: MatchingService, FoundItemReport, LostItemReport, MatchSuggestion, Notification. UC-14: CampusSecurityOfficer, DisputeCase, Claim, VerificationRecord, FoundItemReport. The other lifelines are the use case actors (Item Owner, Finder, Email / Notification Service, Campus Security Officer).
Message names correspond to class operations	Yes	Yes	Yes	Every call message is an operation of the receiving class, with matching parameters. The only other messages are returns, constructors (MatchSuggestion, Notification, VerificationRecord) and sendEmail() to the external Email / Notification Service.
Messages are sent between plausible participants	Yes	Yes	Yes	The actor is the first column and the control object (ItemSearch, MatchingService) the second, as in the lecture slides. No entity object sends a message to a control object; entities only return to it. In UC-14 every call follows an association or dependency in the class diagram (DisputeCase groups Claim, Claim has VerificationRecord, Claim is for FoundItemReport).
Required operations are in the class model, or the model was updated	Yes	Yes	Yes	setFilters(), sortNewestFirst() and ItemReport.matches() (UC-03) and LostItemReport.getContactEmail() (UC-09) were added, and CampusMember.startSearch() was removed. Recorded in Documents/Lab5_Changes.md. UC-14 needed no change.

C. Consistency with the Overall System:

Check	UC-03	UC-09	UC-14	Evidence
The interactions reflect	Yes	Yes	Yes	They use the Lost and Found classes, statuses (WITHDRAWN,

Check	UC-03	UC-09	UC-14	Evidence
the actual proposed system				MATCHED, CLAIMED, PENDING_VERIFICATION) and DFD flows (Delivery Status in UC-09, Item Status Update in UC-14).
The actor is consistent with the selected use case	Yes	Yes	Yes	UC-03: Item Owner. UC-09: Finder, with the Email / Notification Service as secondary actor. UC-14: Campus Security Officer. All match Use_Cases.md.
The sequence of messages is logical	Yes	Yes	Yes	UC-03 sets the filters before running the search and sorts after the loop. UC-09 scores before ranking and ranks before notifying. UC-14 compares the claims before resolving, and approves the winner before rejecting the others.
The diagram is understandable and internally consistent	Yes	Yes	Yes	Messages are numbered in time order, loop frames mark the repeated messages, notes explain the rules, and the message numbers in each notes file match the diagram.

D. Notation required by Exercise 2:

Check	UC-03	UC-09	UC-14	Evidence
A lifeline for every participant	Yes	Yes	Yes	Actors and objects all have lifelines; created objects (MatchSuggestion, Notification, VerificationRecord) start at their creation message.
Messages in the correct order, named after the intended operation, with meaningful parameters	Yes	Yes	Yes	See the message tables in the three notes files.
Return messages where a value comes back	Yes	Yes	Yes	UC-03: 4, 6, 8. UC-09: 3, 10, 15, 17, 18. UC-14: 2, 5, 6, 11, 14, 16.
Activation bars	Yes	Yes	Yes	On every lifeline while it executes, with nested bars for self-calls.
At least one iteration (*) and one guard ([ ])	Yes	Yes	Yes	UC-03: iteration on 3, guard on 5. UC-09: iteration on 2 and 8, guards on 5 and 7. UC-14: iteration on 4 and 12, guard on 12.
Made with a UML tool (draw.io)	Yes	Yes	Yes	The editable .drawio file of each diagram is in Sequence_Diagrams.

E. Mismatches found and fixed:

Diagram	Mismatch	Fix	Fixed by
UC-03, UC-09	Four operations used in the messages were not in the Lab 5 class diagram, and CampusMember.startSearch() let an entity object access a control object.	The class diagram was updated (see Documents/Lab5_Changes.md).	Hamda Albahri (Member 2)
UC-14	The PNG was a screenshot that included a screen tooltip and cut off the diagram frame.	The diagram was exported again from draw.io.	Hamda Bahakeem (Member 3)

Confirmation:

We confirm that all of the consistency checks listed above (A, B and C, and the notation checks in D) were performed on all three sequence diagrams and verified, and that every mismatch found was fixed before submission.

Exercise 4 – Changes Made to Earlier Labs:

Changes made to the Lab 5 class diagram so that it supports the three sequence diagrams.

File	Change Made	Caused By
Project/Object_Modeling/ Class_Diagram/ Consolidated_Class_Diagram	ItemSearch: Add + setFilters(keywords: String, category: String, location: String, dateFrom: Date, dateTo: Date): void	UC-03 message 1: the owner enters the filters (FR-03).
Project/Object_Modeling/	ItemSearch: Add - sortNewestFirst(results:	UC-03 message 7: results are

File	Change Made	Caused By
Class_Diagram/ Consolidated_Class_Diagram	List<FoundItemReport>): List<FoundItemReport>	shown newest first (FR-03).
Project/Object_Modeling/ Class_Diagram/ Consolidated_Class_Diagram	ItemReport: Add + matches(keywords: String, category: String, location: String, dateFrom: Date, dateTo: Date): boolean	UC-03 message 5: the search asks each report whether it fits the filters. It is on ItemReport because category, description, location and eventDate are there.
Project/Object_Modeling/ Class_Diagram/ Consolidated_Class_Diagram	LostItemReport: Add + getContactEmail(): String	UC-09 message 9: the notification copies the owner's contact email (Lab 5 decision).
Project/Object_Modeling/ Class_Diagram/ Consolidated_Class_Diagram	CampusMember: Remove + startSearch(): ItemSearch and the dashed CampusMember ⇢ ItemSearch dependency	In UC-03 the owner works with the ItemSearch control object directly. The lecture slides say entity objects should not access control objects, and CampusMember is an entity.

UC-14 needed no change: its notes file lists no operation missing from the Lab 5 class diagram. All other classes and relationships are unchanged.

Updated Consolidated UML Class Diagram:

